Context-Aware Linguistic Steganography: A Generative Workflow using Large Language Models

NASOUH ALOLABI, Higher Institute for Applied Sciences and Technology, Syria
RIAD SONBOL, Higher Institute for Applied Sciences and Technology, Syria

This paper proposes a black-box linguistic steganography workflow for threaded discussions that hides bits by choosing a reply target and a semantic Angle, then writing an ordinary visible comment. The receiver recovers those choices from a supplied thread snapshot and shared operational parameters. The method does not use invisible characters, and it does not claim token-level or information-theoretic security. On an independent LLM-judge evaluation of 244 unique-output pairs, ZLG was identified in a 10-comment lineup less often (28.3% versus 51.6%), while our method scored higher on register-fit (3.193 versus 2.791, paired sign-test $p = 0.00374$). A separate historical configuration comparison succeeded on 554 of 627 project-side attempts and accepted 304 decode-verified ZLG outputs; recoverable capacity there was 15.14 versus 16.00 bits/comment. Project-side recovery in both artifacts is audit-assisted, so these figures are configuration evidence rather than a blind-receiver ranking.

Additional Key Words and Phrases: Linguistic Steganography, Large Language Models, LLMs, Natural Language Processing, NLP, Black-box Steganography, Semantic Embedding, Agentic Workflows, Imperceptibility

PREPRINT NOTE

This manuscript is a preprint dated September 2, 2026.

1 INTRODUCTION

Linguistic steganography hides a secret inside ordinary text. Early methods mostly swapped synonyms [1]. Natural language has little spare room, so even small statistical quirks can give a detector something to grab [2–4]. Large language models changed the cover: instead of editing an existing sentence, many systems now generate the text [5–7]. Those systems usually hide bits by steering next-token sampling, which can look fluent while still sitting on a modified distribution.

That fluency still has to fit the thread. A model can write a grammatical, low-perplexity sentence that does not belong in the conversation [8–10]. A comment that is fluent but off-topic is conspicuous to a human reader and to a semantic checker.

We hide the payload in two conversational choices rather than in token probabilities. Bits select (i) which comment to reply to in a depth-first flattened thread, and (ii) which *Angle* to talk about from a list both parties rebuild from the public thread and a shared research procedure. A language model then writes an ordinary visible reply for that pair. We do not use invisible characters. Before the comment is kept, the sender runs the same Angle decoder the receiver will use and retries if that decode misses. Figure 1 contrasts token-level embedding with this two-layer selection channel.

The primary contributions of this paper are:

- A two-layer, selection-channel embedding architecture for threaded discussions that encodes bits into reply targeting and Angle selection, with a visible reply rather than a hidden carrier.
- A reconstruction pipeline (thread plus reproducible research) that does not need white-box logits. Reported payload recovery still uses sender-side audit metadata.

Authors' addresses: Nasouh AlOlabi, Higher Institute for Applied Sciences and Technology, Damascus, Syria; Riad Sonbol, Higher Institute for Applied Sciences and Technology, Damascus, Syria.

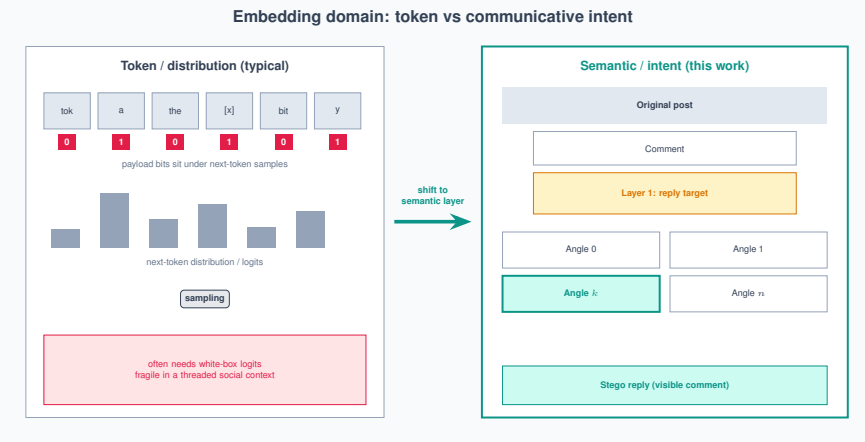

Fig. 1. Conceptual contrast between token-level embedding and our two-layer semantic embedding over a threaded discussion.

- A closed-loop verification mechanism that simulates decoding before the comment is kept and retries synthesis, with an optional revision pass, under stochastic generation.
- An independent LLM-judge comparison on 244 unique-output pairs against the official ZLG baseline, reported together with source-run reliability and a historical 304-pair configuration table, with explicit denominators and without a superiority claim.

2 BACKGROUND

This paper is about hiding that communication is happening, not about encrypting the bytes. **Steganography** embeds a message in an ordinary carrier, such as an image or a piece of text, so an observer has no reason to look for a hidden channel. **Imperceptibility** is the primary objective, not confidentiality as such.

Text is a particularly challenging carrier due to its low redundancy and strict semantic constraints. The classical "Prisoners' Problem" [11] illustrates the goal: two parties, Alice and Bob, must exchange hidden information without alerting a watchful adversary.

Textual steganography methods are typically divided into **format-based** approaches, which exploit layout or structural features, and **linguistic-based** approaches, which modify linguistic form. Within the latter, early techniques such as **synonym substitution** embed bits by altering lexical choices, but suffer from low capacity and high detectability.

Most published generative methods hide bits by biasing next-token sampling. Given a language model and a secret bitstream, the encoder restricts which token may be emitted at each step so that the choice encodes message bits. That process is often called steganographic sampling [12]. It is the literature's token-level model, not the method in Section 4. Security arguments for those systems turn on keeping the modified sampling distribution close to the model's original next-token distribution.

Traditional linguistic approaches offer limited embedding capacity and often leave statistical artifacts. Advances in deep learning and **Large Language Models (LLMs)** now enable generative methods that achieve higher text quality and more secure embedding. Evaluating such systems requires several dimensions of imperceptibility: **perceptual** (human naturalness), **statistical** (distributional similarity to natural text), and **cognitive** (semantic and contextual fidelity) [13].

LLM use in steganography also introduces new sources of detectability. **Hallucination**—where a model generates content that is factually unsupported, semantically inconsistent, or poorly grounded in the preceding context—is one such source. In ordinary text generation, hallucination is mainly a reliability concern. In steganographic generation, it becomes a security concern as well: even a fluent sentence can expose a hidden channel if it contains an unsupported claim, an abrupt topic shift, or a contradiction with the surrounding context. Hallucination therefore threatens **cognitive imperceptibility**, since generated text must remain contextually plausible and communicatively coherent, not merely surface-fluent.

Distinct from hallucination, the **PSIC effect** (Perceptual-Statistical Imperceptibility Conflict) [2] captures a different and arguably more fundamental tension in generative linguistic steganography. Text that is optimized for human fluency is not necessarily statistically indistinguishable from natural human text. For example, selecting only high-probability tokens can produce sentences with low perplexity and strong local readability, but it may also make the generated distribution overly smooth, regular, or concentrated compared with real human writing. Such text can appear natural to readers while remaining detectable to statistical steganalyzers [2].

The PSIC effect therefore shows that **perceptual imperceptibility** and **statistical imperceptibility** are related but not identical goals. Human-written text often contains variation, irregularity, and locally unexpected word choices. By contrast, overly fluent model-generated text may lack this natural variance. Increasing the embedding rate can sometimes force the sampler to select from a broader range of tokens, which may better approximate the diversity of human text; however, this same process can also reduce readability or semantic coherence if the token choices become poorly aligned with the context. Thus, higher capacity does not automatically imply better security. Secure generative steganography requires balancing embedding rate, fluency, distributional similarity, and contextual consistency.

2.1 Large Language Models as Text Generators

Large Language Models (LLMs) predict the next token from a prefix [3, 14, 15]. That is why token-level steganography can ride sampling, and why this paper does not: we never modify the next-token distribution.

2.2 Large Language Models in Generative Linguistic Steganography

Because text produced by modern LLMs can closely resemble ordinary human-authored text, these models provide effective engines for generative text steganography. Rather than editing an existing cover, a growing line of work treats the model itself as a steganographic sampler: the hidden message steers generation, so the stego text is drawn from a realistic communication distribution from the start. The public availability of high-quality models, together with the efficiency of sampling-based embedding, has made this route more practical than earlier designs.

LLMs such as GPT-2 [15], LLaMA [16], and Baichuan2 [17] are common backbones. Most schemes still tie message bits to token choices: at each step the bits pick which candidate is emitted. White-box methods need the exact model and vocabulary, and they change the sampling distribution when they embed [7]. Black-box methods call an API or UI and never see logits. That is the distinction this paper needs. Section 4 is black-box in that sense: the writer is an ordinary generation call. Layer 2 recovery is still an LLM identification prompt, not a logit decoder.

New approaches, such as **LLM-Stega** [7], explore black-box generative text steganography using the user interfaces (UIs) of LLMs, which removes any need to observe internal sampling distributions. Secret bits are encrypted and mapped onto a prearranged keyword set that steers generation, and candidate outputs from which the message cannot be recovered are discarded via reject sampling, preserving both extraction accuracy and semantic quality [7].

Table 1. Comparison of the literature synthesis in this section with prior surveys of text/linguistic steganography.

Survey	Period covered	Methodology	Carrier scope	LLM-era coverage	Contextual-compatibility focus
Majeed et al. [20] (2021)	Pre-transformer	Narrative	Text-only (synonym, spacing, coding)	None	No
Setiadi et al. [21] (2025)	2021–2025	Narrative	Multi-carrier (image, linguistic, 3D mesh)	Yes	No
Wang et al. [4] (2023)	Deep-learning era	Narrative	Detection-side (linguistic steganalysis)	Partial	No
This synthesis	2020–2025	Structured synthesis	Text-only (LLM-based)	Yes	Yes

Co-Stega retrieves recent posts and uses prompts to raise reply entropy, aiming at higher capacity in social-media settings without dropping fluency [8].

Zero-shot linguistic steganography instead relies on in-context learning: covertext samples are supplied in the prompt, and the stego text is produced within a question–answer (QA) exchange, so the output reads as a natural reply rather than as free-standing generated text [6].

The increasing popularity of deep generative models has made it feasible for provably secure steganography to be applied in real-world scenarios, as they fulfill requirements for perfect samplers and explicit data distributions [3, 18, 19].

LLM-based steganographic methods are typically evaluated on two primary axes: imperceptibility (perceptual, statistical, and cognitive measures) and embedding capacity (e.g., bits per token or bits per word). Imperceptibility evaluations may include automatic metrics (PPL, Distinct-n, KL/JSD) as well as human judgements; embedding capacity is usually reported as bits/token or overall embedding rate. Section 3 surveys how the literature applies, evaluates, and constrains these methods in more detail.

3 RELATED WORK

Majeed et al. (2021) [20] surveyed pre-LLM text steganography techniques, predating the current transformer era, emphasizing synonym replacement, spacing manipulation, and coding schemes such as Huffman coding. The methods of that generation often relied on context-free grammars or Markov chains and tended to produce syntactically correct but semantically incoherent cover text. Setiadi et al. (2025) [21] examine more recent AI-powered data hiding across image, linguistic, and 3D mesh carriers (2021–2025), observing that LLMs have "revitalized" linguistic steganography through methods built on GPT-2 [22], GPT-3 [23], LLaMA2 [24], and Baichuan2 [25]. On the detection side, Wang et al. [4] review deep-learning linguistic steganalysis and document how detection techniques have evolved alongside generative hiding methods.

Those surveys cover the field. None of them treats fit-to-thread as the main criterion for LLM-based methods. We do, because that is the channel. The remainder of this section synthesizes 31 primary studies from 2020 to 2025 on what was published, where it was applied, how it was evaluated, how it used external knowledge, and where it still breaks. Table 1 places this synthesis next to the three surveys above.

Table 2. Representative model families across the reviewed studies. Categories are non-exclusive, so totals exceed 100%.

Model Type (Studies)	Models and Representative Works
Open-weight Models (25/31, 80.6%)	GPT-2 [3, 9, 18, 32], LLaMA/LLaMA2 [6, 8, 19, 33], BERT [1, 10, 13, 34–39], OPT [40], BART [1, 5], Qwen [29, 41]
Proprietary Models (7/31, 22.6%)	GPT-3.5/4, ChatGPT [7, 26–28, 42, 43]
Custom or Task-Specific Architectures (5/31, 16.1%)	From-scratch or task-specific models [2, 31, 35, 36, 40]

3.1 State of the Published Literature

The 31 studies span 2020 through 2025, with 10 publications falling in 2020–2023 and 21 in 2024–2025 alone—a sharp acceleration that coincides with the wider availability of large-scale models. Widely used model families—GPT, GPT-2, and LLaMA [15, 16, 23]—appear across both periods, though proprietary systems grow more common in the later studies as black-box access becomes a viable design choice. Across the reviewed corpus, open-weight and research-accessible models appear more frequently than closed commercial systems, reflecting the practical importance of reproducibility and controllable experimentation.

The field has also evolved from earlier white-box token-sampling schemes toward a broader design space that includes hybrid and black-box methods. The reviewed studies now span generative schemes that write stego text from scratch [2, 3, 12, 18], rewriting systems that paraphrase existing covers [5], black-box pipelines that treat the model as an opaque API [7, 26–28], zero-shot protocols driven primarily by prompting [6], context-aware frameworks that retrieve or constrain external information to increase semantic plausibility or payload [8, 9, 29], and methods that emphasize stronger security, robustness, or token-level quality control [3, 18, 30, 31].

Table 2 shows how the reviewed studies distribute across model families. This table is best read as a synthesis of methodological tendencies rather than a strict one-study-one-category classification: many papers combine multiple model families, so the counts overlap and the percentages do not sum to 100%. Open-weight models account for the largest share because they support reproducible experiments and direct control over sampling or decoding, with GPT-2 and LLaMA-family systems appearing especially often. BERT-based methods also recur frequently, but usually as masked-language or scoring components rather than as the sole generation backbone. Proprietary systems such as GPT-3.5/4 are mainly used in black-box or prompt-driven settings, while custom or task-specific architectures appear less often but remain important for studies that prioritize provable security, reversible encoding, or tightly controlled payload design.

3.2 Application Domains

The reviewed studies show that **covert communication** remains the dominant application of LLM-based linguistic steganography. In this setting, secret information is embedded into seemingly benign text so that the message can pass through ordinary communication channels with minimal suspicion. LLM-based methods improve this use case by producing more fluent and contextually plausible outputs than earlier rule-based or local-substitution systems [2, 3, 7].

Within this application space, the reviewed papers explore several operational variants. Some methods generate stego text from scratch using a shared model or sampling strategy [3, 18]. Others rely on paraphrasing or rewriting so that a pre-existing cover text can be transformed while preserving its apparent meaning [5]. Recent black-box and prompt-based approaches such as **LLM-Stega**, **DeepStego**, and multi-criteria linguistic optimization show that covert communication can also be implemented through commercial or hosted interfaces without direct access to token probabilities, using prompt design, keyword sets, stylistic features, and reject-sampling style verification instead

Table 3. Application categories across the reviewed studies. Categories are non-exclusive.

Primary Application	Variant / Category	Representative Studies	Total Count
Covert Communication	Generation-based (from scratch)	[2, 3, 18, 19, 30–32, 35, 37–39]	11
	Paraphrasing, Rewriting, or Editing	[5, 34]	2
	Black-box and Prompt-based	[6, 7, 26–28, 33]	6
Context-Aware Systems	Domain, Emotion, or Social Adaptation	[8–10, 13, 29, 41]	6
Watermarking	Content Attribution (Boundary Cases)	[1, 36, 40, 42–44]	6

[7, 27, 28]. Context-aware systems such as **Co-Stega**, **Hi-Stega**, and emotionally controllable steganography further adapt the generated text to specific domains, topics, entities, emotional tones, or social settings in order to improve plausibility and increase effective payload [8, 9, 29]. This last variant—adapting generation to the surrounding domain and discourse rather than relying on generic fluency—is the closest precedent in the literature to the context-aware, thread-level embedding developed in Section 4.

Although adjacent topics such as watermarking, authorship attribution, or prompt-hiding attacks are relevant to the broader field of information hiding, they remain analytically distinct from covert payload transmission. The 2025 watermarking studies are therefore treated as boundary cases: useful for understanding robustness, semantic preservation, and detection methods, but not interchangeable with steganographic communication systems [44].

A comprehensive breakdown of how the reviewed studies are distributed across these primary application areas and variants is summarized in Table 3.

3.3 Evaluation Practices in the Literature

How published LLM steganography is scored varies by paper, which makes cross-paper comparison hard. Section 6 states the formulas this paper actually computes: recoverable selection-channel width, intention-to-treat versus successful-output-only rates, GPT-2 perplexity on the historical run, and the five LLM-judge criteria. The rest of this subsection is about what the 31 studies reported, not a second copy of those formulas.

3.3.1 Perplexity. Perplexity is an imperceptibility metric for fluency, where lower values generally indicate more natural text [45]. However, its usefulness for language model evaluation is limited. It mainly reflects internal consistency rather than factuality or reasoning, so fluent but implausible statements can still receive low perplexity scores [46]. It is also sensitive to text length, with short sequences often receiving inflated scores even when they are highly fluent [47]. In addition, perplexity is not directly comparable across models unless tokenization and vocabulary are aligned, since these design choices affect the score itself [48]. Fang et al. [47] further argue that token-averaged likelihood can under-represent performance on important low-probability tokens, while Wang et al. [46] note that repetition can sometimes lower perplexity in misleading ways. Taken together, these issues suggest that perplexity should be used cautiously and alongside other measures when assessing steganographic text quality.

3.3.2 Statistical Divergence Metrics. Kullback-Leibler Divergence (KLD) [49] and Jensen-Shannon Divergence (JSD) are information-theoretic metrics used to evaluate steganographic security. KLD quantifies information loss by measuring the relative entropy between cover and stego distributions, serving as the theoretical standard for security modeling despite being asymmetric and failing as a strict distance measure. JSD improves upon this as a symmetric, bounded variant that measures how far each distribution lies from their average, providing a more stable basis for formulating

statistical imperceptibility bounds—particularly when language models approximate human text distributions. Together, these two metrics attempt to capture how closely steganographic outputs mimic legitimate communication channels.

However, real-world application reveals reliability limits, most notably the PSIC effect [2]. KLD and JSD scores can diverge from human judgment as statistical optimization progresses: methods with favorable divergence scores may still produce low-quality text that appears suspicious to human observers. This discrepancy is also dataset dependent—the same method can yield different divergence values on different corpora at equivalent embedding rates (e.g., KLDs of 19.507 on IMDB versus 8.295 on Twitter for VAE-Stega [2]), making cross-paper comparison unreliable. In addition, studies use different formulas, feature spaces, and measurement scales, including latent BERT features versus direct word distributions. Meteor [3], for example, is associated with reported KLD values ranging from 0.045 in one setting to 7.491–11.845 in others. Consequently, these metrics capture distributional distance but not perceptual naturalness, so they should be paired with semantic and human-centered measures.

3.3.3 Capacity Metrics. Capacity is judged by four metrics:

- **Bits per Token (BPT):**
$$\text{BPT} = \frac{\text{Total Secret Bits}}{\text{Total Tokens}} \tag{1}$$
- **Bits per Word (BPW):**
$$\text{BPW} = \frac{\text{Total Secret Bits}}{\text{Total Words}} \tag{2}$$
- **Embedding Rate (ER) [50]:** Average density of hidden information per textual unit
$$\text{ER} = \frac{1}{N}\sum_{i=1}^{N} \text{bits}_i \tag{3}$$
where N is the number of textual units (words, tokens, or sentences) and bits_i is the number of bits embedded in the i-th unit. In some fixed-payload schemes, the total number of embedded bits is predetermined for each stego text and does not depend on its length; for example, LLM-Stega can assign a constant payload to every generated stego sentence, so the embedding rate can also be expressed as $\text{ER}_{\text{fixed}} = B_0$, where B_0 is the fixed number of bits embedded in each stego text.
- **Utilization Rate (UR):**
$$H = -\sum_{x \in \mathcal{X}} P(x) \log_2 P(x) \tag{4}$$
$$\text{UR} = \left(\frac{\text{Actual Bits Embedded}}{H}\right) \times 100\% \tag{5}$$

These quantities describe how densely a secret is embedded, but they are affected by systematic biases that limit cross-system comparison, summarized in Table 4.

Tokenization differences make "1 BPT" from one paper difficult to compare with "1 BPT" from another when the studies use different tokenizers. The PSIC effect shows that higher density can hurt human fluency yet help statistical evasion. Model frequency preferences shrink the real alphabet to high-probability tokens, so naive entropy limits overstate usable space. Ambiguous reporting—practical vs. effective payload, or bit length vs. stego length—can make capacity claims appear stronger than they are. Finally, BPW/BPT ignore semantics, so high values can be reported even when the resulting text is suspicious or incoherent. Recent works reveal additional distortions: loop-count overhead, dictionary-size caps, baseline-dependent "improvements," watermarking goals that invert the desired signal, conversational filler that dilutes BPW, and nonlinear ER curves that make any single threshold difficult to interpret. The

Table 4. Five primary bias categories affecting capacity metrics in steganographic evaluation.

Bias Category	Core Problem	Critical Implication
Tokenization Inconsistencies	Metrics depend entirely on specific tokenizers (e.g., GPT-2 BPE vs. word-level)	Direct comparisons across papers become meaningless when tokenization strategies differ
The PSIC Effect	Conflicts between imperceptibility and statistical security are ignored	High capacity may degrade human fluency while paradoxically improving detection resistance
Model Training Bias	Utilization Rate calculations assume uniform token availability	Actual hiding space is smaller than theoretical entropy due to model frequency preferences
Reporting Ambiguities	No standard definition of "capacity" across systems	Practical payload vs. effective payload distinctions create misleading efficiency claims
Context Blindness	Density metrics treat text as neutral bit containers	Semantic incoherence constitutes a security failure that BPW/BPT fails to penalize

Table 5. Representative capacity calculations reported by selected methods. Full generated passages are omitted because they do not support direct cross-study comparison.

Method	Reported Capacity	Encoding Basis	Comparability Note
Meteor	1280 bits	160-byte payload × 8 bits	Payload total; depends on the selected message length.
LLM-Stega	64 bits per sentence	Fixed semantic-slot mapping	Sentence-level capacity; not directly comparable with BPW or BPT.
GCStego	16 bits per word	Fixed word-level mapping	Mapping-specific rate under the study's candidate construction.
ParaLS	3 bits	Binary lexical watermark	Example payload rather than a normalized embedding rate.
Zero-shot	Token-level BPW/capacity	Generated-token coding	Capacity depends on the representation, model, and prompt protocol; not directly comparable with selection bits.
Discop	968 bits over 200 tokens	4.84 bits per token at $p = 0.95$	Token-level rate tied to the model, tokenizer, and truncation setting.

added 2025 studies reinforce this point: DeepStego emphasizes high capacity and recovery accuracy, and multi-criteria optimization reports reliable stylistic decoding [27, 28], but these results also make cross-paper comparison harder because the reported gains depend on different baselines, prompts, datasets, and decoding assumptions.

To contextualize theoretical capacity metrics, Table 5 summarizes representative embedding capacities across the reviewed methods. Capacity calculations vary significantly based on the encoding mechanism. **Fixed-length mapping** techniques, such as **LLM-Stega** and **GCStego**, derive capacity from rigid keyword or token mappings—64 bits per sentence and 16 bits per word, respectively. **ParaLS** operates similarly but on a smaller scale, encoding a simple 3-bit binary watermark. In contrast, **payload-dependent** methods scale with the cover text or secret message length: **Meteor's** capacity of 1280 bits corresponds to an embedded payload of 160 bytes, **Zero-shot** capacity is directly tied to the Base64 representation of the secret bitstream, and **Discop** uses a statistical approach that, with a truncation parameter of $p = 0.95$, achieves an entropy rate of 4.84 bits per token, yielding 968 bits over a 200-token sequence.

3.3.4 Security Metrics. The sources evaluate security and reliability through the following integrated metrics:

- **Detection Accuracy (Acc) / Error Rate (PE):** Measures a classifier's ability to distinguish between cover and stego text. An accuracy of 50% (or PE of 50%) is the gold standard, indicating the stego text is statistically identical to normal text.
- **F1 Score:** The harmonic mean of precision and recall, used to verify detection reliability, especially when datasets are balanced.
- **Robustness (Attack Resistance):** Evaluated via the Mean Impact of Attack (mIOA) or bit accuracy after removal attempts such as denoising auto-encoding, word substitution, or sentence deletion.
- **False Positive Rate (FPR):** Measures how often human or normal model-generated texts are incorrectly flagged as containing secret messages.

Table 6. Frequency of evaluation metric categories in the reviewed studies. Categories are non-exclusive.

Dimension	Metric Category	Studies	Main Comparability Constraint
Imperceptibility	Perplexity	13	Model, tokenizer, corpus, and sequence length
Imperceptibility	Statistical divergence (KLD/JSD)	10	Estimated distributions and sampling settings
Imperceptibility	Semantic similarity	13	Reference construction and metric choice
Imperceptibility	Human evaluation	1	Protocol, annotator population, and sample size
Security	Detection accuracy or F1	21	Detector, class balance, and train/test data
Capacity	BPW or BPT	27	Tokenization and payload accounting
Robustness	Recovery or attack resilience	5	Attack type, strength, and evaluation protocol
Efficiency	Generation or extraction time	5	Hardware, model size, and implementation

Recent robustness-oriented work broadens this evaluation set. Position-agnostic generation tests perturbation tolerance across multiple attack types, active-attack repair methods evaluate message integrity after tampering, and contrastive semantic watermarking reports F1 under rewriting and substitution attacks [30, 31, 44]. This shift suggests that future evaluations should report not only clean-channel capacity and fluency, but also degradation curves under realistic editing.

3.3.5 Evaluation Challenges and Gaps. The broad range of evaluation metrics across the reviewed studies is summarized in Table 6, which categorizes metrics by type and quantifies their usage frequency. The numbers in that table reveal several persistent gaps. Shared benchmarks are rare: only around 20% of studies use common datasets, which makes cross-paper comparison difficult even when the same metric name appears in both papers. Human evaluation was reported in just one of the 31 reviewed studies—a surprisingly low share given how central perceptual plausibility is to the field's core claims. Robustness testing is similarly sparse, absent in roughly 60% of studies. Many papers also evaluate along only one or two metric dimensions, leaving imperceptibility, security, and capacity to be inferred from partial evidence rather than directly measured together. Taken together, these gaps mean the strongest conclusions available from the reviewed corpus are qualitative rather than quantitative. Section 6 reports an independent LLM-judge comparison together with source-run reliability and a historical configuration table; GPT-2 and unigram divergence are scored on that historical run, not on the 2026-08-15 freeze.

3.4 Integration of External Knowledge Sources

External knowledge integration is a recurring design pattern in the reviewed literature. Across the surveyed studies, external information serves three main purposes: improving contextual plausibility, maintaining topic consistency, and—in some cases—expanding the effective space available for embedding. Table 7 summarizes the main knowledge-integration patterns and their roles.

Rather than relying only on the internal prior of the language model, several recent systems inject external guidance during generation. Retrieved posts, headlines, or comments can provide a realistic discourse context for the cover text [9]. Knowledge-graph triples can constrain the content that should appear in the output, improving semantic consistency over longer spans [33]. Named entities and sentiment labels add a finer-grained control layer when the goal is emotional or conversational consistency [29]. Frequency-based external statistics can also be used to steer generation toward distributions that better match the target channel [32].

Table 7. Representative patterns of external knowledge integration.

Knowledge Type	Typical Role	Primary Benefit	Examples
Semantic Resources	Topic, entity, and emotion guidance	Better semantic control	Co-Stega [8], knowledge-graph guided methods [33, 35], emotionally controllable steganography [29]
Domain Corpora	Domain adaptation	Better stylistic fit	FREmax [32], specialized datasets
Prompt Engineering	In-context control	Better task alignment	Zero-shot [6], DeepStego [27], multi-criteria optimization [28]
Context Retrieval	Evidence or example retrieval	Higher contextual plausibility	Co-Stega [8], retrieval-augmented pipelines [9]

Another recurring pattern is the use of domain-specific corpora. Fine-tuning or conditioning on domain data helps align the generated text with the vocabulary, stylistic conventions, and topical expectations of the intended communication channel. This pattern appears in systems such as VAE-Stega [2], Meteor [3], Hi-Stega [9], FREmax [32], Rewriting-Stego [5], and Summarization-Stego [10]. When additional training is not possible, black-box methods move this domain adaptation into the prompt, style specification, or retrieval stage. Zero-shot approaches provide examples directly in the context window, LLM-Stega frames the generation request with topic-constrained prompts, DeepStego uses prompts and synonym guidance, multi-criteria optimization encodes through stylistic controls, and Co-Stega retrieves recent posts to anchor the generated reply in a plausible local context [6–8, 27, 28]—the same retrieve-then-anchor strategy that underlies the deterministic research stage described in Section 4.2.

Table 8 provides a more detailed breakdown of how external knowledge sources are integrated across the reviewed studies.

These five patterns differ primarily in how directly they constrain the generation process. Structured knowledge approaches [33, 35] use external knowledge graphs or graph attention networks to enforce long-range semantic consistency—a stronger form of control than prompting alone, but one that requires the knowledge resource to exist and be aligned with the target domain. Information retrieval frameworks [8, 9] pull contextually relevant material from external sources at runtime, grounding the cover text in real discourse without requiring model retraining. Linguistic resource methods [1] rely on synonym databases and paraphrase corpora to constrain lexical choice while preserving meaning—a lighter form of intervention that nonetheless depends on the quality and coverage of the external resource. External model feedback is the most common pattern, appearing in seven of the 31 reviewed studies: auxiliary models such as BERT serve as discriminators or soft scorers to assess naturalness or security during generation [37, 40]. Finally, shared side information methods [3, 6] use public models or auxiliary history to synchronize probability distributions between sender and receiver without explicitly passing knowledge through the channel, trading interpretability for coordination efficiency—the same trade-off that motivates the shared, deterministic search pipeline in Section 4.2.

3.5 Limitations and Trade-offs

Across the surveyed studies, the main constraints recur around imperceptibility, effective payload, semantic coherence, tokenization reliability, deployment assumptions, and attack robustness.

3.5.1 The PSIC Effect Revisited. Generative steganography faces an intrinsic tension: optimizing for perceptual fidelity degrades statistical security. Aggressive perplexity minimization produces unnaturally "sharp" distributions that deviate from high-variance human writing, increasing detection vulnerability. Higher embedding rates force selection of lower-probability tokens, degrading text quality but expanding the candidate pool to enhance anti-steganalysis resistance. At

Table 8. Comprehensive overview of external knowledge integration patterns in LLM-based steganography.

Approach	Variant / Category	Representative Studies	Count
Structured Knowledge Integration	Knowledge Graphs (KG) & Graph Attention Networks	Semantic controllable steganography via knowledge graph [33], joint linguistic steganography with BERT and graph attention networks [35]	2
	Information Retrieval (IR): Retrieval-Augmented Generation (RAG) & Corpus Search	Hi-Stega [9], Co-Stega [8], emotionally controllable text steganography [29]	3
Linguistic Resource Integration	Paraphrase DBs, Synonym DBs & Dependency Datasets	Paraphraser-based lexical substitution [1], DeepStego [27], post-hoc watermarking of black-box LLM text [43], group-chat imperceptible text steganography [41]	4
	External Statistical Analysis: Character/Token Frequency Distributions	FREmax [32]	1
External Model Feedback	Auxiliary Scorer, Critic & Discriminator Models	Rewriting-Stego [5], principled natural language watermarking [36], CPG-LS [37], controllable semantic steganography via summarization [10], ALiSa [39], DeepTextMark [40], contrastive semantic watermarking [44]	7
Shared Knowledge / Side Information	Shared Auxiliary Datasets & Public Contexts	Meteor [3], zero-shot generative linguistic steganography [6], reversible data hiding via masked language modeling [34], natural language steganography by ChatGPT [26]	4

elevated capacities, output distributions asymptotically approach natural language's high-entropy chaos, camouflaging signals within legitimate statistical noise [2].

3.5.2 Attack Vulnerability and Security Concerns. Current techniques exhibit a substantial attack surface across multiple vectors, including word-level deletions and lexical substitutions that disrupt dependency parsing. Polishing and re-translation attacks progressively erase watermarks; at high intensities, these structural changes can reduce detection performance to an F1 score of 0.5, rendering the watermark undetectable. Furthermore, adversarial machine learning enables the extraction of secret model parameters from black-box systems, potentially compromising integrity entirely. Recent work responds by moving robustness into the embedding design itself: position-agnostic generation avoids dependence on fixed token locations, active-attack repair models attempt to recover damaged stegotext, and contrastive semantic watermarking tests resilience under rewriting and substitution [30, 31, 44]. To mitigate distributional risks, Variational Auto-Encoders are also employed to learn the statistical distribution of normal text, aiming to reduce the KL divergence between cover and stego distributions and resolve the conflict between perceptual and statistical imperceptibility [43].

3.5.3 Capacity and Contextual Constraints. Short, low-entropy contexts severely constrain embedding capacity. Social media posts and technical documents offer minimal redundancy for hiding [8], while semantic preservation requirements

compete directly with information embedding objectives. Brief texts further lack sufficient contextual support for effective steganographic operations [13]. Newer approaches attempt to recover some capacity through synonym guidance and stylistic feature control, but these gains remain tied to the selected prompt, cover text, or candidate-token space [27, 28].

Early generative methods also produced fluent but semantically arbitrary text, violating cognitive imperceptibility [13]. When generation is conditioned only on local token history, the output may drift away from the source topic or conversational logic, which is especially suspicious in low-entropy channels such as social media replies. This contextual drift worsens when methods must keep generating until an arbitrarily large payload is fully embedded: texts become unnaturally long, coherence decays over time, and practical payload efficiency drops even when reported bit capacity appears high [26, 33, 37, 42].

Our paired comparisons against the zero-shot generative linguistic steganography baseline [6] are reported in Section 6. An independent LLM-judge evaluation on 244 unique-output pairs is the current quality table; a 2026-07-30 configuration comparison reports recoverable capacity and GPT-2 perplexity on 100 post clusters. Both are conditional on accepted outputs, use audit-assisted project-side recovery, and do not establish a security advantage for either method.

3.5.4 Segmentation and Tokenization Barriers. Subword tokenization introduces critical extraction ambiguities that are fundamental to white-box steganographic methods. Byte-pair encoding splits words unpredictably, creating multiple valid segmentations that corrupt message boundaries and cause decoding failures. These ambiguities impose hard capacity caps regardless of algorithmic sophistication—SparSamp suffers accuracy degradation from token ambiguity, while ShiMer cannot effectively boost entropy due to tokenization constraints [19].

This segmentation ambiguity persists even when sender and receiver employ identical LLMs and tokenizers. Because modern LLMs decompose words into subword units, and vocabularies frequently contain both complete words and their constituent components, any given text string admits multiple valid token representations. This multiplicity is compounded by the detokenization-retokenization gap: senders must convert tokens to plain text to conceal their transmission, while receivers must subsequently retokenize this text to recover hidden messages. Despite shared tokenization rules, receivers may select different token sequences than senders intended—a sender might encode a message using two separate tokens (“any” and “thing”), only for the receiver to retokenize the surface string as the single token “anything”. Because LLMs operate autoregressively, a single divergent token choice alters the probability distribution for all subsequent predictions, causing total decoding failure for the remainder of the message. To address this white-box-specific challenge, researchers propose SyncPool, a mechanism that aggregates ambiguous tokens into pools and employs synchronized random number generators to ensure both parties select identical tokens [19]. Because this work embeds payload through reply targeting and Angle selection rather than through raw token identity (Section 4.4.1, Section 4.4.2), it does not inherit that detokenization gap. Layer 1 is a discrete parent index. Layer 2 is recovered by a semantic shortlist and an identification prompt, not by retokenizing a steganographic sampler’s output.

3.5.5 White-box versus Black-box Trade-offs. The architectural choice between white-box and black-box access paradigms involves fundamental trade-offs. Across the reviewed corpus, only a minority of studies operate primarily through black-box or prompt-driven access (Table 3), while the majority employ white-box or hybrid methods. White-box methods typically assume access to internal language-model components such as token probabilities, decoding states, vocabularies, or model parameters, and some approaches additionally require expensive fine-tuning or retraining

procedures to align the model distribution with steganographic objectives. While these methods can achieve strong statistical imperceptibility through distribution-aware generation (e.g., VAE-Stega and arithmetic-coding-based methods), they suffer from reduced accessibility, high computational cost, and deployment complexity.

In contrast, black-box approaches operate through standard LLM APIs or user interfaces without requiring direct access to model internals or retraining, substantially improving practicality and usability. However, the distinction is primarily about *model access*, not the absence of probabilistic coordination, as several black-box approaches still rely on shared probabilistic mechanisms, semantic distributions, or synchronized auxiliary structures between sender and receiver. Recent semantic black-box methods also prioritize robustness against paraphrasing, token perturbation, and noisy communication channels rather than purely token-level imperceptibility. Among the few peer-reviewed black-box methods currently available, LLM-Stega reports a capacity of 5.93 bpw with perplexity 165.76, illustrating that improved accessibility does not eliminate the PSIC trade-off between payload capacity, text quality, robustness, and security. Several recent systems further blur strict categorical boundaries by combining retrieval-, generation-, editing-, or substitution-based paradigms within hybrid steganographic frameworks.

3.5.6 Computational and Resource Constraints. Performance optimization conflicts directly with computational efficiency [3]. Superior results demand increased resources and memory, particularly for large models with external knowledge bases [18]. Real-time latency constraints limit optimization depth, while performance degradation emerges at operational scale. Variable length sampling, rejection sampling, active repair, and token-level quality control significantly increase encoding or decoding complexity, and LLM-based steganography requires 3x to 5x more time than prior methods [19]. The Meteor system exemplifies hardware constraints, running nearly 50× slower on mobile devices compared to GPU environments. LLM-specific deployment obstacles compound these costs: hallucinations and nonsensical continuations can corrupt the covert bitstream or create detectable artifacts, while lossy transmission, editing, or platform-side normalization can irreversibly damage extraction. As model quality improves, steganalysis systems also improve, producing an arms race in which better generation does not automatically translate into better security.

3.5.7 Critical Unresolved Challenges. Foundational gaps persist across the field: theoretical security guarantees remain unestablished, resilience to advanced attacks is limited, evaluation frameworks lack standardization, responsible development guidelines are absent, non-English performance lags substantially, and real-world deployment testing remains insufficient. Several studies also surface unresolved concerns around reversibility, bias, and misuse potential, indicating that robust engineering alone will not be enough without clearer security and governance models [1, 6, 34, 51].

Quantitative evidence underscores several of these constraints. The PSIC effect creates a trade-off where increasing the embedding rate (e.g., to ∼5 bpw) strengthens anti-steganalysis but significantly degrades perceptual text quality [2]. Attack vulnerability is notable; for example, adaptive attacks can reduce bit accuracy to near-random levels (∼0.6) [36]. Segmentation ambiguity and entropy variability in communication channels are major impediments that lead to decoding failures and restricted practical capacity [3]. Finally, computational optimizations induce significant overhead, with some reordering algorithms causing up to 69% performance degradation in processing speed [18].

3.6 Contextual Compatibility as a Research Direction

The nearest precedents are context-aware systems such as Co-Stega and Hi-Stega, which retrieve or constrain external information so the cover fits a domain. This paper uses that context as the payload channel, not as extra fluency: bits choose a reply target and an Angle, and the generated comment is an ordinary reply in the thread (Section 4).

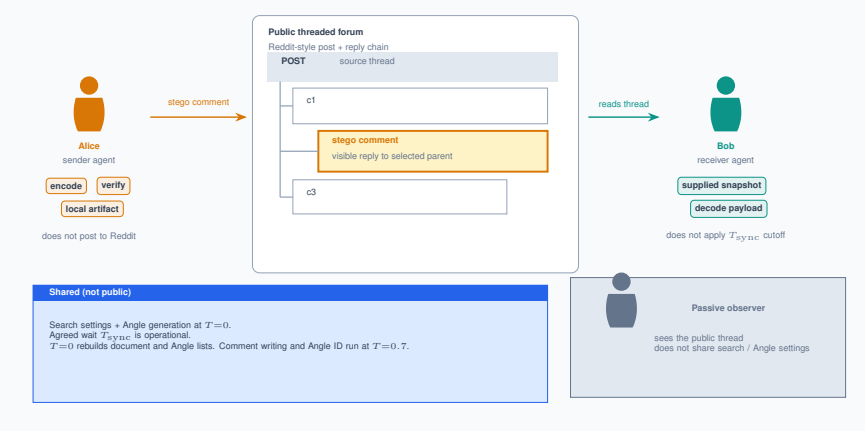

Fig. 2. Covert communication scenario: Alice and Bob use a public threaded channel. They share search and Angle-generation settings, and they agree on a temporal window. The live receiver still needs that snapshot to be supplied; it does not apply the cutoff itself.

4 METHODOLOGY

We propose a black-box steganographic method that hides a secret message inside a public conversation. The sender and the receiver both have accounts on a social media platform. The sender finds a thread, writes what looks like a normal comment or reply, and publishes it. The receiver later reads that same thread, finds the sender's contribution, and recovers the hidden payload. Our study uses Reddit-style data as the main case. The design targets any threaded discussion environment; whether it transfers cleanly to other platforms remains to be evaluated. Figure 2 shows the end-to-end covert scenario and the shared assumptions needed for synchronization.

4.1 What Alice and Bob share

Both parties need accounts on the same threaded platform. They also share the search settings, the model used to generate search terms and Angles at `temperature=0`, and an agreed wait T_{sync} between Alice's snapshot and the moment she publishes. Temperature 0 applies to reconstructing the document list and the Angle list. It does not apply to writing the comment or to the Angle discriminator, both of which run at the shared stego temperature (0.7 in the balanced profile used for the reported runs).

4.2 Building a shared document list

Alice's agent picks a busy thread. It then builds a document list that Bob can rebuild: the post, the comments Alice saw, and pages retrieved by a shared search procedure. Figure 3 summarizes that research stage.

Search terms come from the post title, body, and URL, not from the comment section. A temperature-0 prompt asks for at least 12 unique queries and aims for 15–20. The balanced profile then keeps 8 of them. Each kept query is sent with a fixed request shape (`first=1`, `count=10`). The live backend tries Google Custom Search first, then other engines if that path is empty. Duplicate links and PDFs are dropped, surviving URLs are fetched in batches of three, and the pipeline records hashes of the term list and of the fetched corpus so the same inputs can be replayed.

From that corpus the Angle runner, still at temperature 0, extracts an ordered list of Angles. Each Angle is a category, a source quote, and a tangent: the direction the reply should talk about. The reported runs cap the list at 32. Angle

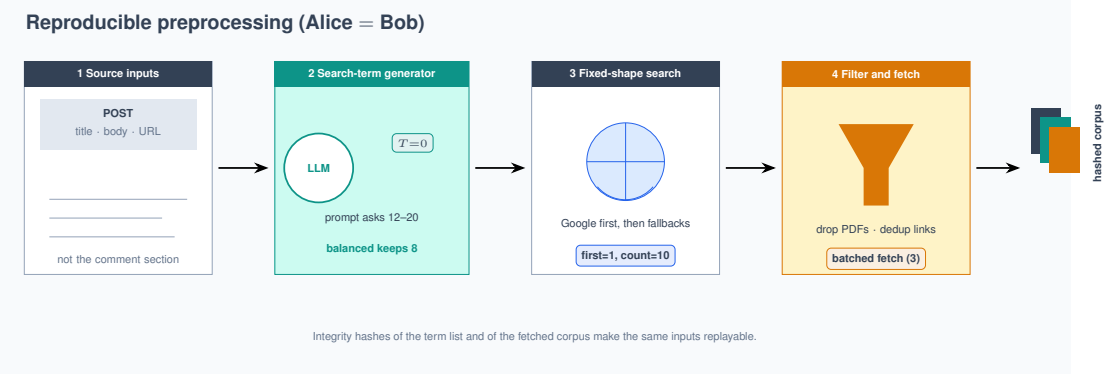

Fig. 3. Research stage: temperature-0 term generation from title, body, and URL; a fixed search request shape; filtering and batched fetching; integrity hashes for replay.

generation is a prior stage. The single-comment encoder does not rebuild the list after choosing a reply target. The reported artifacts used a context-weighted sampler once per post at preparation time, usually with no selected parent. They did not rebuild Angles after Layer 1 at encode time.

4.3 Waiting to publish

Alice needs time to research, pick a reply target, pick an Angle, and write a comment. If new comments arrive in that interval, Bob cannot be sure which parents Alice saw. The intended fix is an agreed duration T_{sync}: Alice snapshots the thread at t_{start} and publishes at $t_{publish}$; Bob keeps comments with time at most $t_{publish} - T_{sync}$. Figure 4 shows that intended window.

The live receiver does not apply this cutoff. It locates Alice's comment, deletes that subtree from a supplied post object, and rebuilds from what it was given. Supplying a snapshot that already matches Alice's view is an operational duty of the caller. The reported encode/decode workloads skip even that rebuild: they decode against the sender's already-angled post.

4.4 Two choices carry the payload

The payload is not hidden in invisible characters, and it is not hidden by biasing next-token sampling. Bits choose two ordinary visible things: which comment Alice replies to, and which Angle she talks about. A language model then writes a short public reply for that pair. Figure 5 shows the split.

4.4.1 Which comment to reply to. The nested comment tree is flattened with a deterministic depth-first walk, so both parties number the same comments the same way. There are $C = n + 1$ reply targets: index 0 is a top-level reply to the post, and indices 1 through n are the flattened comments. If the index is not 0, the encoder also builds the parent chain of that comment so the writing prompt can see the local thread.

4.4.2 Which Angle to talk about. The Angle list from the research stage is flattened and given sequential `idx` values. The next bits pick one index. At writing time that Angle's category, source quote, and tangent are interpolated into the prompt together with the post, the parent chain, and a research excerpt matched to the source quote.

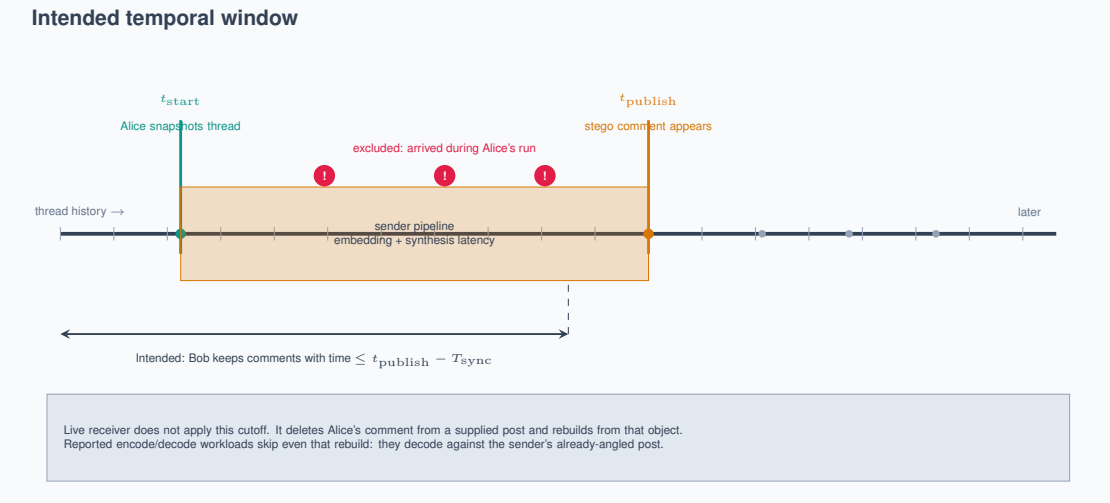

Fig. 4. Intended temporal window: Bob would keep comments dated at most $t_{publish} - T_{sync}$. The live receiver does not apply this filter; it rebuilds from the supplied post after deleting Alice's comment.

4.4.3 How many bits those choices can carry. Both layers consume a prefix of the same bitstream produced in Section 4.7. Layer 1 reads the first w_1 bits; Layer 2 reads the next w_2.

Two widths must not be mixed. The *physical* widths are the field sizes the single-comment encoder actually writes: $w_1 = W(n)$ and $w_2 = W(A-1)$, where $W(m) = \lceil \log_2(m+1) \rceil$ for $m > 0$. If the integer from those bits is larger than the last valid choice, the encoder reduces it modulo the choice count. Generation still succeeds, but several bit patterns then land on the same parent or the same Angle, so those patterns cannot be inverted.

The *recoverable* widths drop that aliasing: $\lfloor \log_2 C \rfloor$ and $\lfloor \log_2 A \rfloor$. A thread with 8 reply targets and 32 Angles therefore carries $3 + 5 = 8$ uniquely decodable bits in one comment. Capacity claims in Section 6 use these recoverable widths.

The live single-comment encoder writes the physical widths and wraps with modulo. A separate lossless writer restricts each field to the recoverable width, then stores that value at the physical width. That injective path is used by the multi-frame planner in Section 4.8, not by the single-comment path that produced the reported tables.

Leftover bits beyond $w_1 + w_2$ are reported as unembedded. They are not dropped, and they are not in the visible comment. Recovering the full compressed string from one comment then needs sender-side metadata: audit-assisted recovery, which is what the reported workloads use.

4.5 Writing the comment and checking that Bob can read it

A nonzero-temperature model writes a JSON array of exactly three short reply candidates. The balanced profile uses temperature 0.7. The prompt sees the post, the parent chain, the selected Angle, and a research excerpt. The Angle is a routing hint, not a label the comment should announce.

Before anything is treated as published, the sender runs the same Angle decoder Bob will use, in strict mode. A candidate is accepted only if that decoder returns the selected Angle index and a contextuality gate passes. If none of the three candidates is accepted, the encoder retries with the same parent and the same Angle; the balanced budget is 6 retries after the first attempt. On the last attempt it may also revise a near-miss with a contextual revision prompt

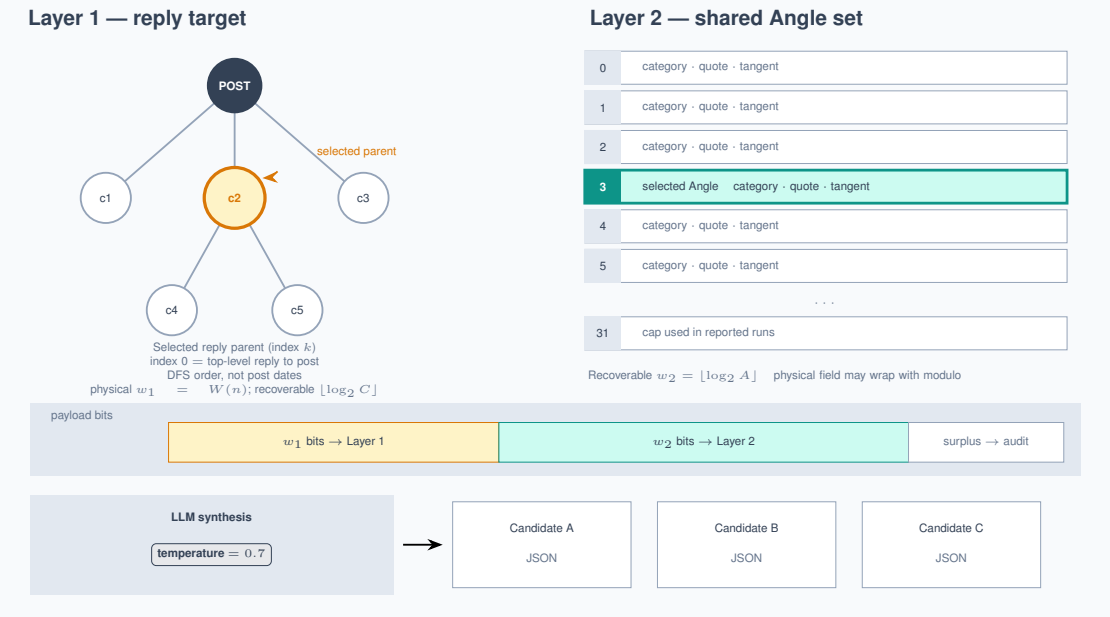

Fig. 5. Two-layer embedding: Layer 1 selects a reply target in a depth-first flattened thread; Layer 2 selects an Angle from a shared list; an LLM then writes a visible reply for that pair.

rather than sampling a fresh triple. The live sender writes a local artifact. It does not post to Reddit. Figure 6 shows the verify loop next to Bob's recovery path.

4.6 How Bob recovers the bits

Layer 1 is public: Bob sees which comment Alice replied to, and that parent maps back to the DFS index.

Layer 2 is not printed as a number. The full reconstruction path locates Alice's comment by author, deletes it from the supplied post, optionally rebuilds research and Angles, then:

(1) embeds the comment with a sentence transformer (`all-MiniLM-L6-v2`) and shortlists the nearest Angles, default 20 after a pool of up to 120;
(2) reranks that shortlist with a mix of cosine similarity and lexical overlap;
(3) asks the same stego model, at temperature 0.7, to return a single line `idx: N` naming the canonical Angle index.

If the model returns a 1-based rank into the shortlist, that rank is mapped back. If nothing parses, the top semantic candidate is used. Sender acceptance requires a strict parse and rejects those fallbacks. The reported workloads call the decoder in non-strict mode, pass the compressed bitstring from the encode audit, and may treat the audit's Angle index as authoritative when it disagrees with the discriminator.

4.7 Preparing a text payload

If the secret is text, it is first optionally wrapped by a protection transform, then compressed against a session dictionary built from the post body, the search-result texts, and the flattened comment bodies, in that order. The default transform on the balanced profile is `plain`: the payload is unchanged. Two other modes exist, `hmac_xor_v1` and `secure_compact_v2`,

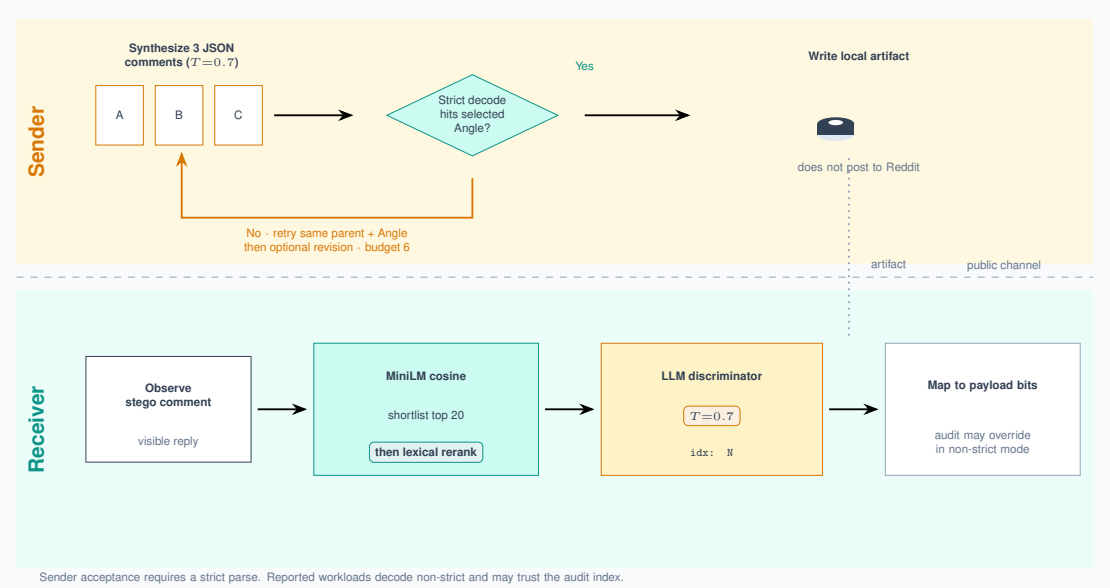

Fig. 6. Sender-side verification retries synthesis until a strict decode hits the selected Angle. The receiver recovers Layer 2 with a cosine shortlist of 20, a lexical rerank, and a temperature-0.7 `idx: N` prompt.

which add a keyed stream cipher and a truncated MAC; they are recorded on the sender audit so Bob applies the matching inverse. They add confidentiality on top of the covert channel. They are not what makes the channel covert.

The compressor emits a one-bit mode flag, then either raw UTF-8 bits or a dictionary parse of literals and pointers. It keeps the dictionary parse only when that parse is strictly shorter, so the worst-case expansion is one bit. Literal runs are capped at 250 characters; pointers are used only for matches of three characters or more. Field widths and the backward parse that chooses the token sequence are defined once in the shared codec; Section 5 states the token layout. This compressor dictionary is not the same object as the Angle-input sampler. Figure 7 sketches the two encoding modes.

The first bit Layer 1 reads is therefore that mode flag. A payload that does not fit in $w_1 + w_2$ physical bits leaves a surplus in the audit, as in Section 4.4.3.

4.8 When one comment is not enough

A longer payload can be split across several frames: each frame is one reply under one post. That planner is implemented. It is not the pipeline behind the reported LUCID and ZLG tables, which used the single-comment encoder plus audit-assisted decode.

On the multi-frame path the payload is protected, then compressed against an empty dictionary, which forces the UTF-8 mode of Section 4.7. An Elias-gamma header then stores the frame count F and the payload bit length. Each slot is sized by recoverable capacity from Section 4.4.3, the last slot is zero-padded, and the receiver rejects a stream whose declared F or padding is wrong. This path uses the lossless writer, so the bits-to-selection map is injective. If the sampler is context-weighted, this is also the path that rebuilds Angles after resolving the parent of each frame. The receiver does not discover frames in the public thread; the caller must supply an ordered list of comment references.

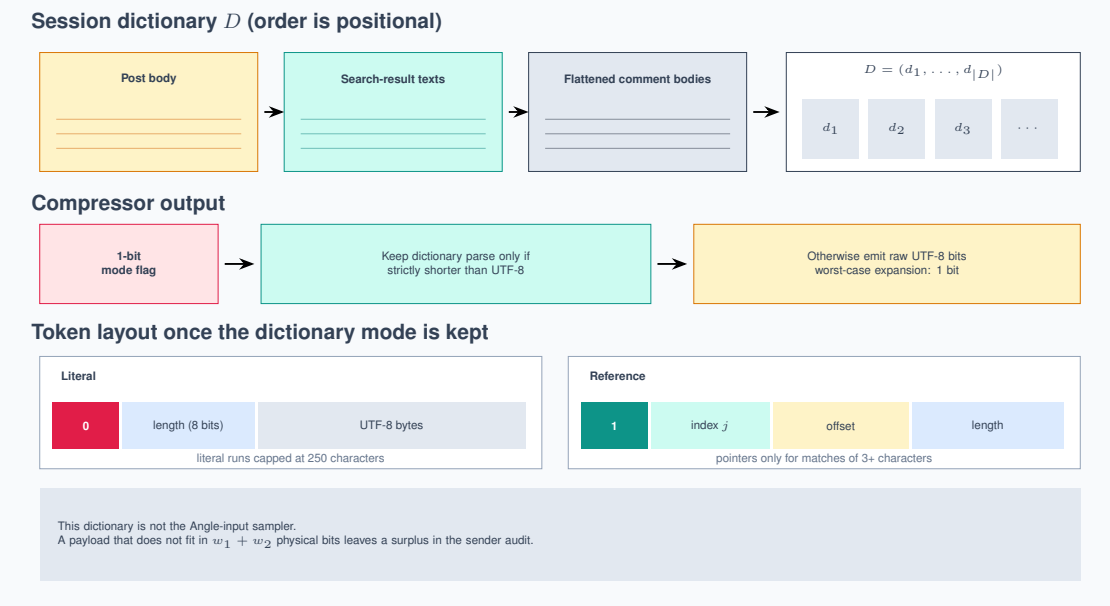

Fig. 7. Payload compression builds a session dictionary from thread and research text, then keeps a dictionary parse only when it is strictly shorter than UTF-8.

5 IMPLEMENTATION

The runnable pipeline is five stages: data load, research, Angle generation, stego encoding, and decode. Each stage writes artifacts to disk so later stages can reuse the same corpus and hashes. Search-term generation and Angle extraction run at `temperature=0` from the same inputs. Comment writing and Angle identification do not: both use the shared stego temperature (0.7 on the balanced profile).

5.1 Pipeline stages

- Data load fetches unresolved URL content and replaces the post body with the extracted text.
- Research generates search terms from the post title, body, and URL at temperature 0, trims them to the profile cap (8 on balanced), queries a fixed-shape search backend with fallbacks after Google, drops duplicate links and PDFs, and fetches page text in batches of three. It stores the flat search-results corpus and integrity hashes.
- Angle generation flattens the chosen input documents and extracts triplets of category, source quote, and tangent. Long inputs are split into 30k-character segments and batched up to 80k characters by default. If the backend path fails, the pipeline falls back to a direct LLM prompt with the same JSON schema. The reported runs used a context-weighted sampler at this stage, once per post.
- Encoding protects and compresses the payload, spends bits on a reply target and an Angle, writes three candidate comments, and keeps a candidate only if a strict local decode hits the selected Angle and a contextuality gate passes.

Table 9. Token layout of the compressed payload stream. Every field has a fixed width given the session dictionary D, but decoding is sequential: a token's start position depends on where the previous token ended.

Token	Field layout	Cost (bits)
Literal	`0` ‖ length ℓ (8 bits) ‖ UTF-8 bytes	$9 + 8b$
Reference	1 ‖ index j ($W(\lvert D\rvert)$ b) ‖ offset ($W(\lvert d_j\rvert)$ b) ‖ length ($W(L_{\max})$ b)	$1 + W(\lvert D\rvert) + W(\lvert d_j\rvert) + W(L_{\max})$

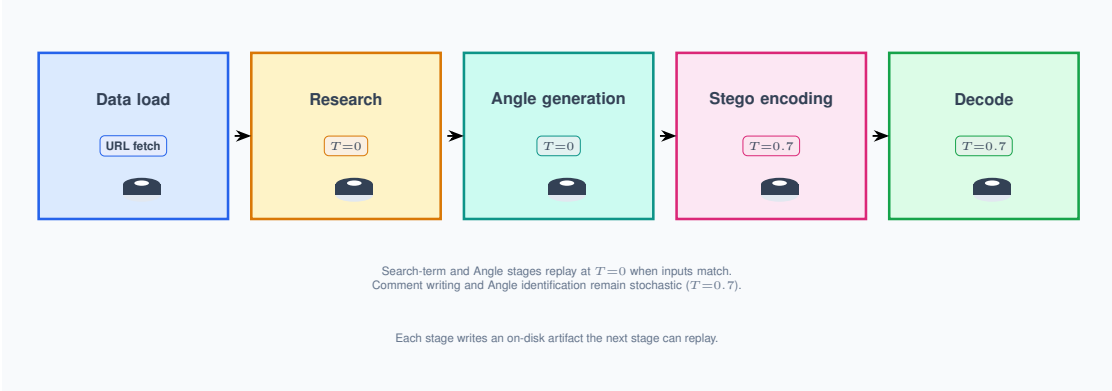

Fig. 8. Implementation stages and persisted artifacts. Search-term and Angle stages replay at temperature 0 when the inputs match; comment writing and Angle identification remain stochastic.

- Decode shortlists Angles with MiniLM cosine similarity, reranks with lexical overlap, and asks for `idx: N` at the stego temperature. The full reconstruction path can rebuild research and Angles; the reported workloads decode against the sender's already-angled post and pass the audit bitstring.

Figure 8 aligns these stages with the on-disk artifacts.

5.2 Shared codec

The bit layer in Sections 4.7–4.8 lives in one module of pure functions. Dictionary construction for compression, the field-width function W, the token grammar, and the physical-versus-recoverable width rules each have one definition there. The sender and receiver call that module rather than reimplementing it, so round-trip tests can drive the codec with fixtures and no model.

The pipelines around it still own parent lookup, Angle regeneration, semantic search, the discriminator, audit override, and revision. They are not limited to I/O.

Three configuration choices change the bit layer, so both sides must agree: the payload transform (`plain`, `hmac_xor_v1`, or `secure_compact_v2`), the pre-shared secret required by the keyed modes, and whether the compressor's dictionary capacity profile is on. The sender records the transform in the run audit. The compressor capacity profile is off by default because dictionary indices are positional.

Table 9 is the token layout used by the dictionary parse. A literal is flag `0`, an 8-bit character length, then UTF-8 bytes. A reference is flag 1 plus index, offset, and length fields sized by W. Decoding is sequential: a token's start depends on where the previous token ended.

The single-comment encoder writes physical widths and wraps overflow with modulo. The multi-frame planner uses the lossless writer that stores only recoverable widths.

5.3 Model roles

Exact model IDs are configuration. The roles are not:

- Search-term generation and Angle extraction run at `temperature=0` so both parties can rebuild the same lists from the same inputs.
- Comment writing runs at nonzero temperature (0.7 on balanced) so retries produce different wording for the same parent and Angle.
- Angle identification uses MiniLM cosine similarity, a default shortlist of 20 with lexical rerank, then the same stego temperature for an `idx: N` prompt. It is not a temperature-0 oracle.

6 EVALUATION

This section reports two dated comparisons against the official zero-shot generative linguistic steganography (ZLG) baseline. They do not share a denominator, and they do not measure the same thing.

The current quality table is an independent LLM-judge evaluation on 244 unique-output pairs drawn from the 2026-08-15 LUCID versus ZLG freeze. Generation and acceptance rates for that source run sit beside it. The second table is a historical configuration comparison from 2026-07-30: 304 accepted pairs, then averaged over 100 source posts. Neither comparison is a capacity-matched, blind-receiver, decision-grade ranking.

6.1 How the numbers are computed

Recoverable capacity for our method is the uniquely decodable selection-channel width,

$$\lfloor \log_2 C \rfloor + \lfloor \log_2 A \rfloor,$$

where C is the number of reply targets (flattened comments plus the post itself) and A is the number of flattened Angles. When a count is not a power of two, leftover index states are not counted. On the 2026-08-15 freeze this is the field stored as encoded payload bits for our method. ZLG's displayed bits are useful payload bits on successful hides in a capacity-matched setting, not ZLG's native token-channel maximum.

Reliability uses two denominators. Intention-to-treat counts every attempted sample, including failures. Successful-output-only scores, including the judge table, are computed only on accepted generations. Failed attempts stay in the attempt counts.

GPT-2 perplexity is $\exp(\sum \text{NLL} / \sum \text{scored tokens})$ per text, then averaged. Lower means more GPT-2-predictable, not more human. The 2026-08-15 freeze skipped GPT-2, KL, JSD, BLEU, ROUGE, and BERTScore, so those columns appear only in the historical table.

Word count on the 2026-08-15 freeze uses the publication tokenizer. The historical table used a narrower token pattern, so the two word-count rows are not interchangeable.

The independent LLM judge is not G-Eval. G-Eval is an older 1–5 quality path with a matched human reference. The judge here is a Codex Luna, high-reasoning run of five blinded criteria on already-generated comments. Slates are seeded. The model is not deterministic. Scores are not a reader study, and they are not proof of authorship or security.

6.2 Independent LLM judge

The source freeze is 1,608 complete pairs from 339 posts. The judge subset keeps, in sorted pair order, the first pair per post whose ZLG comment is globally unique in that pool: 244 pairs from 244 posts, 244 unique texts on each arm.

Table 10. Independent LLM-judge result on 244 unique-output post pairs (Codex Luna, high reasoning). Lower is better for standout, weakest-text, and suspicion AUROC. Higher is better for attribution and register-fit. Chance is 10%, 33%, 0.5, and 25% for the first four rows.

Criterion	Our method	ZLG	Note
Standout detection	126/244 (51.6%)	69/244 (28.3%)	ZLG less often picked in the 10-way slate
Weakest text picked	71/244 (29.1%)	116/244 (47.5%)	Human control 57/244 (23.4%)
Suspicion AUROC vs human	0.661	0.650	No clear raw-score difference ($p = 0.385$)
Thread attribution	89.3%	75.0%	Descriptive only
Register-fit (1–5)	3.193	2.791	+0.402 post-cluster; 106/67/71; $p = 0.00374$

Ninety-five posts with no unique ZLG output were excluded. The five criteria reuse those pairs. They are not 2,928 independent generated samples. All 2,928 tasks completed with no task errors.

Standout detection. One generated comment is inserted into nine human comments from the same post. The judge names the inserted slot. Chance is 10%. Lower is better.

Weakest text. One slate holds a human comment, our comment, and the ZLG comment. The judge names the weakest of the three. Chance is 33%. Lower is better. The three pick rates share the same 244 slates and sum to 100%.

Suspicion AUROC.. Each arm, including a held-out human comment, receives a 0–100 suspicion score. The table cell is Mann–Whitney AUROC of that method's scores against the human arm, with ties counted as 0.5. Chance is 0.5. Lower is closer to the human distribution. The reported $p = 0.385$ is a sign test on paired raw scores, not a test of the AUROC gap.

Thread attribution. The judge picks which of four thread blurbs a comment belongs to. Chance is 25%. Higher means stronger thread-specific linkage. This row is descriptive: there is no paired sign test in the scoring artifact.

Register-fit. Using five genuine subreddit comments as examples, the judge scores style on a 1–5 `overall` field. The table cell is the arithmetic mean of that field. Higher is better. This is the only criterion with a claimed paired post-cluster sign test.

ZLG is harder to pick out of a same-thread lineup. Our comments are named weakest less often than ZLG's, though still more often than the human control. Register-fit favors our method on the paired sign test. Suspicion AUROC is close for both arms. None of this is a security result, and none of it uses the 244 unique-text count from the 2026-07-30 run.

6.3 Reliability of the 2026-08-15 source run

Do not mix these denominators with Table 10.

The LUCID campaign that fed the comparison requested 4,044 samples and recorded 2,543 successes and 1,501 failures (846 generation failures, 613 Angle mismatches, 42 invalid JSON). Every recorded success used audit-assisted compressed-bitstring recovery. That is an engineering check, not blind recovery from a public thread.

The ZLG freeze that produced the 1,608-pair source file attempted 1,711 hides, accepted 1,610, and decode-verified 1,603 (96 quality-gate failures, 5 leakage-check failures, 0 harness skips). Our paired rows in that freeze are also 1,608, all marked audit-assisted. A later completed ZLG file in the same run directory processed all 2,543 LUCID successes and accepted 2,340. The judge did not use that larger file.

Table 11. Historical post-clustered paired results over 100 source posts (2026-07-30). Lower is better for GPT-2 perplexity. Lexical quality is the run's 0–100 index, not a raw MATTR column. BERTScore F1 is the unrescaled figure from that report.

Metric	Our method	ZLG	Paired result
Recoverable capacity (bits/comment)	15.14	16.00	ZLG higher; Holm-adjusted sign-test $p < 0.001$
GPT-2 perplexity	68.15	53.70	ZLG lower; $p = 0.0001$
Lexical quality index (0–100)	99.36	98.45	Ours higher; $p = 0.0015$
Word count	22.4	17.3	Ours longer; $p < 0.001$
BERTScore F1	0.842	0.848	Comparable in that report

On the 1,608-pair freeze, post-clustered means over 339 posts give recoverable encoded bits of 13.42 per comment for our method and 13.90 for ZLG, with ZLG higher on 230 posts (Holm-adjusted sign-test $p = 1.49 \times 10^{-14}$). Mean word counts are 23.86 versus 15.91. The lexical-quality index is 99.37 versus 96.93. GPT-2 perplexity and unigram KL/JSD were not scored on this freeze.

6.4 Historical configuration comparison

Table 11 is the 2026-07-30 `context_weighted_v2` run, not the 2026-08-15 freeze. Our method attempted 627 samples and succeeded on 554 (88.4%). Failures were 70 Angle mismatches and 3 generation failures. ZLG was fed those 554 successes in a capacity-matched setting with EGS parameters (threshold = 0.01, temperature = 0.7, alpha = 1.0, max_bpw = 2). The original writeup counts 304 accepted of 554 (54.9%), with 250 failures: 134 quality-gate, 94 cover-sentence extraction, 22 prompt leakage. A 2026-07-31 recode of the same run treats the 94 extraction skips as harness faults, leaving 460 server invocations, 304 accepted (66.1%), and 156 genuine ZLG failures (133 quality-gate, 22 leakage, 1 HTTP 500). Keep both accountings. The metric table is successful-output-only, then averaged over 100 source posts.

The Angle list was capped at 32, so the angle component contributed at most 5 recoverable bits per frame. Project-side rows used audit-assisted compressed-bitstring recovery. The research pool was 167 posts, 35 of them sampled ten times, and 244 of the 304 our-method texts were unique. That 244 is not the judge subset.

On that configuration ZLG leads on recoverable channel width and GPT-2 perplexity. Our replies are longer and score slightly higher on the lexical-quality index. The capacity gap is narrow because the Angle pool was capped at 32. It is not a general upper bound on either system.

6.5 What these comparisons do not show

The judge result is about the unique-ZLG-output subset, one pair per post. The historical table is one angle-capacity configuration and one baseline service configuration, with reused posts and audit-assisted project-side recovery. KL/JSD, synthetic detection, and grouped passive-detector ROC-AUC are computed by later runners and are not in the 2026-08-15 freeze. A dedicated steganalysis and editing-attack evaluation remains future work. This paper does not claim a security advantage for either method.

7 CONCLUSION

This work hides a payload in a threaded discussion by choosing a reply target and an Angle, then asking an LLM to write an ordinary comment. Capacity is whatever those two lists uniquely decode. The design does not insert bits into next-token sampling, and it does not need white-box logits.

An independent LLM-judge run on 244 unique-output pairs found ZLG harder to pick in a same-thread lineup (28.3% versus 51.6% standout detection) and our method higher on register-fit (3.193 versus 2.791, paired sign-test $p = 0.00374$).

The source campaign succeeded on 2,543 of 4,044 LUCID attempts, all with audit-assisted recovery. A historical 2026-07-30 configuration, post-clustered over 100 posts, had recoverable capacity 16.00 versus 15.14 bits/comment and GPT-2 perplexity 53.70 versus 68.15, with 554 of 627 project-side attempts succeeding. Those two artifacts answer different questions. Neither is a blind, capacity-matched ranking, and neither establishes resistance to an adaptive steganalyst. Future work should freeze a symmetric manifest, score GPT-2 and divergence on that freeze, decode without sender audit files, and test dedicated steganalysis and editing attacks.

REFERENCES

[1] Jipeng Qiang, Shiyu Zhu, Yun Li, Yi Zhu, Yunhao Yuan, and Xindong Wu. Natural language watermarking via paraphraser-based lexical substitution. *Artificial Intelligence*, 317:103859, 2023.

[2] Zhong-Liang Yang, Si-Yu Zhang, Yu-Ting Hu, Zhi-Wen Hu, and Yong-Feng Huang. Vae-stega: linguistic steganography based on variational auto-encoder. *IEEE Transactions on Information Forensics and Security*, 16:880–895, 2020.

[3] Gabriel Kaptchuk, Tushar M Jois, Matthew Green, and Aviel D Rubin. Meteor: Cryptographically secure steganography for realistic distributions. In *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security*, pages 1529–1548, Virtual Event, Republic of Korea, 2021. ACM.

[4] Yihao Wang, Ru Zhang, Yifan Tang, and Jianyi Liu. State-of-the-art advances of deep-learning linguistic steganalysis research. In *2023 International Conference on Data, Information and Computing Science (CDICS)*, page 20–24. IEEE, December 2023. doi: 10.1109/cdics61497.2023.00014. URL http://dx.doi.org/10.1109/CDICS61497.2023.00014.

[5] Fanxiao Li, Sixing Wu, Jiong Yu, Shuoxin Wang, BingBing Song, Renyang Liu, Haoseng Lai, and Wei Zhou. Rewriting-stego: generating natural and controllable steganographic text with pre-trained language model. In *International Conference on Database Systems for Advanced Applications*, pages 617–626. Springer, 2023.

[6] Ke Lin, Yiyang Luo, Zijian Zhang, and Ping Luo. Zero-shot generative linguistic steganography. *arXiv preprint arXiv:2403.10856*, 2024.

[7] Jiaxuan Wu, Zhengxian Wu, Yiming Xue, Juan Wen, and Wanli Peng. Generative text steganography with large language model. In *Proceedings of the 32nd ACM International Conference on Multimedia*, pages 10345–10353, Melbourne, Australia, 2024. ACM.

[8] Guorui Liao, Jinshuai Yang, Kaiyi Pang, and Yongfeng Huang. Co-stega: Collaborative linguistic steganography for the low capacity challenge in social media. In *Proceedings of the 2024 ACM Workshop on Information Hiding and Multimedia Security*, pages 7–12, Baiona, Spain, 2024. ACM.

[9] Huili Wang, Zhongliang Yang, Jinshuai Yang, Yue Gao, and Yongfeng Huang. Hi-stega: A hierarchical linguistic steganography framework combining retrieval and generation. In *International Conference on Neural Information Processing*, pages 41–54. Springer, 2023.

[10] Ruifan Zhang, Jianyi Liu, and Ru Zhang. Controllable semantic linguistic steganography via summarization generation. In *ICASSP 2024-2024 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pages 4560–4564. IEEE, 2024.

[11] Gustavus J Simmons. The prisoners' problem and the subliminal channel. In *Advances in Cryptology: Proceedings of Crypto 83*, pages 51–67, Boston, MA, 1984. Springer.

[12] Siyu Zhang, Zhongliang Yang, Jinshuai Yang, and Yongfeng Huang. Provably secure generative linguistic steganography. In *Findings of the Association for Computational Linguistics: ACL-IJCNLP 2021*, pages 3046–3055, 2021.

[13] Changhao Ding, Zhangjie Fu, Zhongliang Yang, Qi Yu, Daqiu Li, and Yongfeng Huang. Context-aware linguistic steganography model based on neural machine translation. *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, 32:868–878, 2023.

[14] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.

[15] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners. Technical report, OpenAI, 2019.

[16] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, et al. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*, 2023.

[17] Aiyuan Yang, Bin Xiao, Binyuan Wang, Binxin Zhang, Ce Bian, Chao Yin, Chenxu Lv, Da Pan, Dian Wang, Dong Yan, et al. Baichuan 2: Open large-scale language models. *arXiv preprint arXiv:2309.10305*, 2023.

[18] Jinyang Ding, Kejiang Chen, Yaofei Wang, Na Zhao, Weiming Zhang, and Nenghai Yu. Discop: Provably secure steganography in practice based on distribution copies. In *2023 IEEE Symposium on Security and Privacy (SP)*, pages 2238–2255, San Francisco, CA, USA, 2023. IEEE.

[19] Yuang Qi, Kejiang Chen, Kai Zeng, Weiming Zhang, and Nenghai Yu. Provably secure disambiguating neural linguistic steganography. *IEEE Transactions on Dependable and Secure Computing*, 2024. Early Access.

[20] Mohammed Abdul Majeed, Rossilawati Sulaiman, Zarina Shukur, and Mohammad Kamrul Hasan. A review on text steganography techniques. *Mathematics*, 9(21), 2021. ISSN 2227-7390. doi: 10.3390/math9212829. URL https://www.mdpi.com/2227-7390/9/21/2829.

[21] De Rosal Ignatius Moses Setiadi, Sudipta Kr Ghosal, and Aditya Kumar Sahu. Ai-powered steganography: Advances in image, linguistic, and 3d mesh data hiding – a survey. *Journal of Future Artificial Intelligence and Technologies*, 2(1):1–23, Apr. 2025. doi: 10.62411/faith.3048-3719-76. URL https://faith.futuretechsci.org/index.php/FAITH/article/view/76.

[22] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners. *OpenAI Blog*, 2019. URL https://d4mucfpksywv.cloudfront.net/better-language-models/language_models_are_unsupervised_multitask_learners.pdf.

[23] T. Brown, B. Mann, N. Ryder, M. Subbiah, J. D. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, et al. Language models are few-shot learners. *Advances in Neural Information Processing Systems*, 33:1877–1901, 2020.

[24] Zhiheng Xi, Wenxiang Chen, Xin Guo, Wei He, Yiwen Ding, Boyang Hong, Ming Zhang, Junzhe Wang, Senjie Jin, Enyu Zhou, et al. The rise and potential of large language model based agents: A survey. *Science China Information Sciences*, 68(2):121101, 2025.

[25] Jianfei Xiao, Yancan Chen, Yimin Ou, Hanyi Yu, Kai Shu, and Yiyong Xiao. Baichuan2-sum: Instruction finetune baichuan2-7b model for dialogue summarization. In *2024 International Joint Conference on Neural Networks (IJCNN)*, pages 1–8. IEEE, 2024.

[26] Martin Steinebach. Natural language steganography by chatgpt. In *Proceedings of the 19th International Conference on Availability, Reliability and Security*, pages 1–9. ACM, 2024.

[27] Oleksandr Kuznetsov, Kyrylo Chernov, Aigul Shaikhanova, Kainizhamal Iklassova, and Dinara Kozhakhmetova. DeepStego: Privacy-preserving natural language steganography using large language models and advanced neural architectures. *Computers*, 14(5):165, 2025.

[28] Kamil Woźniak, Marek R. Ogiela, and Lidia Ogiela. Multi-criteria linguistic optimization for covert communication in secure LLM-based steganography. *Applied Soft Computing*, 185:113960, 2025.

[29] Hao Shi, Wenpu Guo, and Shaoyuan Gao. Emotionally controllable text steganography based on large language model and named entity. *Technologies*, 13(7):264, 2025.

[30] Shuo Lin, Zhenyang Shen, Xiang Li, Jiacheng Fan, and Sixing Wu. Position-agnostic probabilistic generation for robust steganographic text. In *Proceedings of the 34th ACM International Conference on Information and Knowledge Management*, pages 4950–4954. ACM, 2025.

[31] Yingquan Chen, Qianmu Li, Xiaocong Wu, and Zijian Ying. Research on making two models based on the generative linguistic steganography for securing linguistic steganographic texts from active attacks. *Symmetry*, 17(9):1416, 2025.

[32] Kaiyi Pang, Minhao Bai, Jinshuai Yang, Huili Wang, Minghu Jiang, and Yongfeng Huang. Fremax: A simple method towards truly secure generative linguistic steganography. In *ICASSP 2024-2024 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pages 4755–4759. IEEE, 2024.

[33] Yihao Li, Ru Zhang, Jianyi Liu, and Qi Lei. A semantic controllable long text steganography framework based on llm prompt engineering and knowledge graph. *IEEE Signal Processing Letters*, 2024.

[34] Xiaoyan Zheng, Yurun Fang, and Hanzhou Wu. General framework for reversible data hiding in texts based on masked language modeling. In *2022 IEEE 24th International Workshop on Multimedia Signal Processing (MMSP)*, pages 1–6. IEEE, 2022.

[35] Changhao Ding, Zhangjie Fu, Qi Yu, Fan Wang, and Xianyi Chen. Joint linguistic steganography with bert masked language model and graph attention network. *IEEE Transactions on Cognitive and Developmental Systems*, 16(2):772–781, 2023.

[36] Zhe Ji, Qiansiqi Hu, Yicheng Zheng, Liyao Xiang, and Xinbing Wang. A principled approach to natural language watermarking. In *Proceedings of the 32nd ACM International Conference on Multimedia*, pages 2908–2916. ACM, 2024.

[37] Lingyun Xiang, Jiali Xia, Yangfan Liu, and Yan Gui. Cpg-ls: Causal perception guided linguistic steganography. *IEEE Signal Processing Letters*, 30: 1762–1766, 2023.

[38] Siyu Zhang, Zhongliang Yang, Jinshuai Yang, and Yongfeng Huang. Linguistic steganography: From symbolic space to semantic space. *IEEE Signal Processing Letters*, 28:11–15, 2020.

[39] Biao Yi, Hanzhou Wu, Guorui Feng, and Xinpeng Zhang. Alisa: Acrostic linguistic steganography based on bert and gibbs sampling. *IEEE Signal Processing Letters*, 29:687–691, 2022.

[40] Travis Munyer, Abdullah All Tanvir, Arjon Das, and Xin Zhong. Deeptextmark: a deep learning-driven text watermarking approach for identifying large language model generated text. *Ieee Access*, 12:40508–40520, 2024.

[41] Fanxiao Li, Ping Wei, Tingchao Fu, Yu Lin, and Wei Zhou. Imperceptible text steganography based on group chat. In *2024 IEEE International Conference on Multimedia and Expo (ICME)*, pages 1–6. IEEE, 2024.

[42] Zhenyu Xu, Ruoyu Xu, and Victor S Sheng. Beyond binary classification: Customizable text watermark on large language models. In *2024 International Joint Conference on Neural Networks (IJCNN)*, pages 1–8. IEEE, 2024.

[43] Jifei Hao, Jipeng Qiang, Yi Zhu, Yun Li, Yunhao Yuan, Xiaocheng Hu, and Xiaoye Ouyang. Robust and semantic-faithful post-hoc watermarking of text generated by black-box language models. *Frontiers of Computer Science*, 19(9):199357, 2025.

[44] Jianxin Wang, Xiangze Chang, Chaoen Xiao, and Lei Zhang. A contrastive semantic watermarking framework for large language models. *Symmetry*, 17(7):1124, 2025.

[45] Fred Jelinek, Robert L Mercer, Lalit R Bahl, and James K Baker. Perplexity—a measure of the difficulty of speech recognition tasks. *The Journal of the Acoustical Society of America*, 62(S1):S63–S63, 1977.

[46] Yequan Wang, Jiawen Deng, Aixin Sun, and Xuying Meng. Perplexity from plm is unreliable for evaluating text quality. *arXiv preprint arXiv:2210.05892*, 2022.

[47] Lizhe Fang, Yifei Wang, Zhaoyang Liu, Chenheng Zhang, Stefanie Jegelka, Jinyang Gao, Bolin Ding, and Yisen Wang. What is wrong with perplexity for long-context language modeling? In *International Conference on Learning Representations*, volume 2025, pages 94541–94563, 2025.

[48] Abby Morgan. Perplexity for LLM evaluation. Comet ML Blog, November 2024. URL https://www.comet.com/site/blog/perplexity-for-llm-evaluation/. Accessed: 2025-12-31.

[49] S. Kullback and R. A. Leibler. On information and sufficiency. *The Annals of Mathematical Statistics*, 22(1):79–86, 1951. ISSN 00034851. URL http://www.jstor.org/stable/2236703.

[50] Christian Cachin. An information-theoretic model for steganography. In David Aucsmith, editor, *Information Hiding*, pages 306–318, Berlin, Heidelberg, 1998. Springer Berlin Heidelberg. ISBN 978-3-540-49380-8.

[51] Emily M Bender, Timnit Gebru, Angelina McMillan-Major, and Shmargaret Shmitchell. On the dangers of stochastic parrots: Can language models be too big? In *Proceedings of the 2021 ACM conference on fairness, accountability, and transparency*, pages 610–623, Virtual Event, Canada, 2021. ACM.